



---

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

# Informe de Propuesta de Arquitectura

## Caso Semestral: Innovatech Solutions

*Plataforma Inteligente para la Gestión Integral de Proyectos Tecnológicos*

---

DSY1106 — Desarrollo Fullstack III

Sección 1: Diseño de Arquitectura y Patrones de Microservicios (Parcial 1)

Desarrollado por: Najeeb Escobar Pérez

Fecha: Abril 2026

## 1. Introducción

Innovatech Solutions es una empresa que se dedica al desarrollo de software a medida y consultoría tecnológica. Tiene más de 120 empleados. Los equipos son multidisciplinarios: hay gente de backend, frontend, DevOps, diseñadores UX y gestores de proyecto, todos distribuidos en distintas ubicaciones geográficas.

El problema que enfrenta la empresa hoy es bastante claro: no tienen visibilidad en tiempo real de cómo van los proyectos, la asignación de recursos se hace de forma manual y poco eficiente, y los directivos no cuentan con indicadores centralizados que les permitan tomar decisiones con datos concretos. En simple, están operando un poco a ciegas en varios frentes.

Lo que presento en este informe es mi propuesta de arquitectura basada en microservicios para resolver estos problemas. Voy a describir los componentes que diseñé, las tecnologías y versiones seleccionadas, los patrones que elegí y por qué tomé cada decisión. También incluyo una evaluación ética del diseño, siguiendo lo que revisé en clase con el documento Ethically Aligned Design de IEEE.

## 2. Stack Tecnológico y Plataforma de Despliegue

Antes de entrar a la arquitectura en sí, vale la pena dejar claro el stack que elegí y por qué. Al principio consideré varias opciones, pero al final me quedé con tecnologías que tienen soporte de largo plazo y que son estándar en la industria.

### 2.1 Versiones de Tecnología

| Componente         | Tecnología / Versión                | Justificación                                                                                                                      |
|--------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Backend (MS + BFF) | Spring Boot 3.2.x / JDK 21 LTS      | JDK 21 es la versión LTS más reciente. Spring Boot 3.2 trae soporte nativo para virtual threads y es compatible con Jakarta EE 10. |
| API Gateway        | Spring Cloud Gateway 4.1.x          | Integración nativa con Spring ecosystem. Soporte para rate limiting, circuit breaker y JWT validation.                             |
| Frontend SPA       | React 18.3 + Vite 5.x               | React por su ecosistema maduro y pool de desarrolladores. Vite como bundler por su velocidad.                                      |
| Persistencia       | Spring Data JPA 3.2 / Hibernate 6.4 | Abstracción JPA con Repository Pattern. Hibernate como implementación estándar.                                                    |

|                   |                                |                                                                                                    |
|-------------------|--------------------------------|----------------------------------------------------------------------------------------------------|
| Base de Datos     | PostgreSQL 16                  | BD relacional madura, open source, buen rendimiento para operaciones transaccionales y analíticas. |
| Resiliencia       | Resilience4j 2.2.x             | Reemplazo moderno de Hystrix (en mantenimiento). Más liviano, diseñado para Java 8+.               |
| Service Discovery | Spring Cloud Eureka 4.1.x      | Registro dinámico de instancias para escalado horizontal sin reconfiguración.                      |
| Autenticación     | Keycloak 24.x (OAuth2/JWT)     | IdP open source con soporte completo para OAuth2, OpenID Connect y gestión de roles.               |
| Monitoreo         | Prometheus 2.51 + Grafana 10.4 | Estándar de facto para monitoreo en Kubernetes. Scrape de /actuator/prometheus.                    |
| Orquestación      | Kubernetes 1.29 (OKE)          | Oracle Kubernetes Engine en Oracle Cloud. Autoescalado, self-healing, rolling updates.             |
| Notificaciones    | JavaMailSender + Gmail SMTP    | Incluido en Spring Boot (spring-boot-starter-mail). Gmail SMTP gratis hasta 500 correos/día.       |

## 2.2 Plataforma de Despliegue: Oracle Kubernetes Engine (OKE)

Decidí desplegar todo en Oracle Kubernetes Engine (OKE) sobre Oracle Cloud Infrastructure. Cada microservicio se empaqueta como imagen Docker y se despliega como un Deployment con su respectivo Service en Kubernetes. El API Gateway queda expuesto vía Ingress Controller, y los microservicios internos solo son accesibles dentro del cluster.

La gracia de usar Kubernetes es que el autoescalado horizontal (HPA) permite agregar o quitar pods según la demanda, y el self-healing reinicia automáticamente pods que fallen. Además, como se revisó en la sesión de Estrategias y Herramientas con Microservicios, Kubernetes es la plataforma de orquestación más utilizada en la industria para este tipo de arquitecturas.

## 2.3 Monitoreo con Prometheus y Grafana

Cada microservicio expone métricas vía el endpoint /actuator/prometheus (gracias a Spring Boot Actuator + Micrometer). Prometheus 2.51 hace scrape de estos endpoints periódicamente y almacena las métricas en su base de datos de series temporales. Grafana 10.4 se conecta a Prometheus como datasource y muestra dashboards con métricas clave: uso de CPU y RAM por pod, latencia de peticiones, tasa de errores HTTP (4xx, 5xx), número de instancias activas y estado de los circuit breakers.

Además, configuraré AlertManager para enviar alertas cuando alguna métrica supera umbrales críticos (por ejemplo, si la latencia p99 supera los 2 segundos o si un circuit breaker pasa a estado Open).

### 3. Visión General de la Arquitectura

---

La arquitectura sigue un flujo directo: los usuarios acceden a una SPA en React 18.3, que se comunica con el API Gateway (Spring Cloud Gateway 4.1). El Gateway valida JWT contra Keycloak 24.x, aplica rate limiting y enruta al BFF (Spring Boot 3.2). El BFF orquesta las llamadas a los tres microservicios de dominio, protegidas por Circuit Breaker (Resilience4j 2.2). Todo desplegado en OKE con monitoreo Prometheus.

[\*Diagrama adjunto: C4 System Context — archivo: C4\\_01\\_System\\_Context.html\*](#)

[\*Diagrama adjunto: C4 Container — archivo: C4\\_02\\_Container.html\*](#)

#### 3.1 Los Tres Módulos de Dominio

- **Gestión de Proyectos (Spring Boot 3.2 / JDK 21 LTS / PostgreSQL 16):** Core operativo. Maneja el ciclo de vida completo de los proyectos: creación, tareas, asignación de responsables, estados y avance. Decidí que este servicio fuera independiente porque es el que más va a crecer a medida que Innovatech sume clientes.
- **Gestión de Recursos y Colaboración (Spring Boot 3.2 / JDK 21 LTS / PostgreSQL 16):** Gestiona la disponibilidad (capacity) del recurso humano, asignaciones a proyectos y visibilidad entre equipos. Este componente envía notificaciones vía JavaMailSender + Gmail SMTP cuando hay cambios en las asignaciones.
- **Monitoreo y Analítica (Spring Boot 3.2 / JDK 21 LTS / PostgreSQL 16):** Dashboards con KPIs para directivos. La gracia de esto es que consume datos de los otros dos servicios vía API REST interna, no duplica información.

### 4. Justificación de los Patrones de Diseño

---

Cada patrón que elegí responde a un problema concreto. No los seleccioné por moda. Como se revisó en el PPT de Evaluación del Diseño de Arquitectura de Microservicios, la selección debe estar alineada con las necesidades reales del proyecto.

Para cada patrón incluyo un sub-diagrama dedicado que muestra cómo se implementa dentro de la arquitectura.

| Patrón             | Qué hace                                                           | Por qué lo elegí                                                      | Tecnología                          | Diagrama                                         |
|--------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------|--------------------------------------------------|
| Repository Pattern | Capa de abstracción entre lógica de negocio y BD.                  | Permite cambiar de BD sin tocar negocio. Facilita mocks para testing. | Spring Data JPA 3.2 / PostgreSQL 16 | <a href="#">C4_03_Patron_Repository.html</a>     |
| Factory Method     | Crea DTOs según el rol del usuario sin acoplar a clases concretas. | El BFF necesita vistas diferentes por rol. Extensible.                | Spring Boot 3.2 / JDK 21            | <a href="#">C4_04_Patron_FactoryMethod.html</a>  |
| Circuit Breaker    | Envuelve llamadas inter-servicios. Tres estados.                   | Aísla fallos. Si Analítica cae, Proyectos sigue.                      | Resilience4j 2.2 / Prometheus       | <a href="#">C4_05_Patron_CircuitBreaker.html</a> |
| API Gateway        | Punto de entrada único: auth, routing, rate limiting.              | Frontend conoce 1 URL. Seguridad centralizada.                        | Spring Cloud GW 4.1 / Keycloak 24.x | <a href="#">C4_06_Patron_APIGateway.html</a>     |
| BFF                | Orquesta llamadas y agrega datos para el frontend.                 | En vez de 5 llamadas, el frontend hace 1.                             | Spring Boot 3.2 / Arquetipo Maven   | <a href="#">C4_07_Patron_BFF.html</a>            |

#### 4.1 Repository Pattern

En cada microservicio definí las entidades JPA y los repositorios de forma separada. El Service Layer depende de la interfaz Repository, no de la implementación JPA (Hibernate 6.4). Esto me permite mockear repositorios en pruebas unitarias con JUnit 5 + Mockito, y si mañana hay que migrar de PostgreSQL a otra BD, no se toca código de negocio.

**Diagrama adjunto:** *Sub-diagrama Repository Pattern* — archivo: [C4\\_03\\_Patron\\_Repository.html](#)

#### 4.2 Factory Method

La fábrica de DTOs vive en el BFF. Llegar una petición, el BFF identifica el rol del usuario desde el token JWT, y la fábrica decide qué DTO instanciar. Si mañana Innovatech necesita un nuevo rol, solo hay que crear un nuevo DTO y registrarlo en la fábrica. No se toca el endpoint. Open/Closed Principle en la práctica.

**Diagrama adjunto:** *Sub-diagrama Factory Method* — archivo: [C4\\_04\\_Patron\\_FactoryMethod.html](#)

### 4.3 Circuit Breaker

Lo implementé con Resilience4j 2.2 en el BFF. Los tres estados son: Closed (todo funciona), Open (servicio caído, se retorna fallback), Half-Open (se prueba recuperación). Las métricas del circuit breaker se exportan a Prometheus para monitorear en Grafana cuándo un servicio tiene problemas.

Al final del día, que los dashboards estén caídos 5 minutos es molesto pero no crítico; que los equipos no puedan registrar avances sí lo es.

**Diagrama adjunto:** *Sub-diagrama Circuit Breaker con estados y Prometheus* — archivo: [C4\\_05\\_Patron\\_CircuitBreaker.html](#)

### 4.4 API Gateway

Spring Cloud Gateway 4.1 centraliza autenticación JWT (validando contra Keycloak 24.x), rate limiting, routing path-based y balanceo de carga. El frontend React solo conoce una URL base. Prometheus hace scrape de las métricas del Gateway para monitorear latencia y errores.

**Diagrama adjunto:** *Sub-diagrama API Gateway* — archivo: [C4\\_06\\_Patron\\_APIGateway.html](#)

### 4.5 BFF (Backend For Frontend)

Construido como arquetipo Maven en Spring Boot 3.2. En vez de que la SPA haga 5 llamadas a diferentes microservicios, el BFF las hace internamente (con Circuit Breaker), agrega los datos con Factory Method y devuelve una respuesta optimizada.

**Diagrama adjunto:** *Sub-diagrama BFF* — archivo: [C4\\_07\\_Patron\\_BFF.html](#)

## 5. Decisiones Arquitectónicas Clave

---

1. **BD por servicio (PostgreSQL 16 independiente por MS):** Sé que tres bases de datos agrega complejidad, pero el desacoplamiento vale la pena.
2. **REST síncrono + Circuit Breaker:** Simple, compatible con Spring Boot 3.2. Evalué mensajería asíncrona pero para el volumen actual me pareció sobredimensionado.
3. **Despliegue en OKE (Kubernetes 1.29):** Cada servicio es una imagen Docker desplegada como Deployment. HPA para autoescalado, self-healing, rolling updates.
4. **Monitoreo con Prometheus + Grafana:** Spring Boot Actuator + Micrometer expone métricas. AlertManager avisa cuando algo anda mal.
5. **Auth centralizada con Keycloak 24.x:** JWT validado una sola vez en el Gateway. Los MS internos confían en el token ya validado.
6. **Notificaciones con JavaMailSender + Gmail SMTP:** Incluido en Spring Boot, sin dependencias externas de pago. 500 correos/día gratis.

## 6. Evaluación Ética del Diseño

---

Siguiendo los principios del documento Ethically Aligned Design de IEEE (que traduje), y según los criterios del PPT de Evaluación del Diseño de Arquitectura de Microservicios, mi diseño incorpora consideraciones éticas. Innovatech maneja datos de más de 120 personas.

### 6.1 Protección de Datos y Privacidad

HTTPS/TLS en todas las comunicaciones. Control de acceso por roles vía Keycloak 24.x. Minimización de datos: cada microservicio solo accede a lo que necesita. Analítica trabaja con métricas agregadas.

### 6.2 Transparencia y Trazabilidad

Logging centralizado con trazabilidad distribuida. Cualquier KPI es auditable. Las métricas de Prometheus permiten rastrear el comportamiento del sistema en cualquier momento.

### 6.3 Resiliencia Ética

El Circuit Breaker tiene una dimensión ética: preferir mostrar "datos no disponibles" a mostrar datos erróneos que afecten decisiones sobre personas. Las métricas del CB en Prometheus permiten saber exactamente cuándo un servicio estuvo degradado.

## 6.4 Sostenibilidad

El escalado horizontal en OKE permite ajustar pods a la demanda real. Prometheus monitorea el consumo de recursos para optimizar costos.

## 6.5 Accesibilidad

Frontend React 18.3 con estándares WCAG 2.1. Responsivo para equipos distribuidos.

## 6.6 Alineamiento con el Cliente

Cada módulo resuelve un desafío real. No hay tecnología porque sí.

## 7. Índice de Diagramas Adjuntos

---

Todos los diagramas están en formato XML importable en draw.io y en HTML, para abrir los enlaces del informe desde el navegador:

- **C4\_01\_System\_Context.xml** — C4 System Context. Actores, sistema principal con stack y OKE, sistemas externos, Prometheus.
- **C4\_02\_Container.xml** — C4 Container. Detalle interno: Frontend, Gateway, BFF, 3 MS, 3 BDs, Service Discovery, Circuit Breaker, Prometheus+Grafana.
- **C4\_03\_Patron\_Repository.xml** — Sub-diagrama Repository Pattern.
- **C4\_04\_Patron\_FactoryMethod.xml** — Sub-diagrama Factory Method.
- **C4\_05\_Patron\_CircuitBreaker.xml** — Sub-diagrama Circuit Breaker.
- **C4\_06\_Patron\_APIGateway.xml** — Sub-diagrama API Gateway.
- **C4\_07\_Patron\_BFF.xml** — Sub-diagrama BFF.

## 8. Conclusión

---

La arquitectura que propongo combina patrones probados (Repository, Factory Method, Circuit Breaker, API Gateway, BFF) con un stack tecnológico moderno y estable (Spring Boot 3.2, JDK 21 LTS, React 18.3, PostgreSQL 16), desplegado en Oracle Kubernetes Engine y monitorizado con Prometheus y Grafana.